React.js & MySQL

200 Technical Interview Questions & Answers

Each answer is structured across three concise, professional points.

SECTION 1: React.js (Questions 1–100)

Q1. What is React.js?

1. React.js is an open-source JavaScript library developed by Facebook for building user interfaces, particularly single-page applications (SPAs).

2. It follows a component-based architecture where the UI is broken into reusable, self-contained components that manage their own state.

3. React uses a Virtual DOM to efficiently update only the changed parts of the real DOM, resulting in high performance and a smooth user experience.

Q2. What is the Virtual DOM in React?

1. The Virtual DOM is an in-memory, lightweight JavaScript representation of the actual DOM tree that React maintains internally.

2. When state or props change, React first updates the Virtual DOM, then uses a diffing algorithm (Reconciliation) to compute the minimal set of changes needed.

3. Only the identified differences are applied to the real DOM, minimising costly reflows and repaints and improving rendering performance.

Q3. What are React components?

1. React components are independent, reusable pieces of UI that accept input (props) and return React elements describing what should appear on screen.

2. Components can be defined as JavaScript functions (Functional Components) or ES6 classes (Class Components), with functional components being the modern standard.

3. Each component encapsulates its own logic, state, and rendering, promoting separation of concerns and code reusability across the application.

Q4. What is JSX?

1. JSX (JavaScript XML) is a syntax extension for JavaScript that allows developers to write HTML-like markup directly inside JavaScript code.

2. Babel transpiles JSX into React.createElement() calls at build time, so browsers can execute it as regular JavaScript.

3. JSX improves readability and developer experience by colocating UI structure with component logic while still supporting full JavaScript expressions inside curly braces {}.

Q5. What are props in React?

1. Props (properties) are read-only inputs passed from a parent component to a child component to customise its behaviour or appearance.

2. They follow a unidirectional data flow (top-down), ensuring a predictable data model and making components easier to debug and test.

3. Props can carry any JavaScript value including strings, numbers, objects, arrays, and callback functions, enabling rich communication between components.

Q6. What is state in React?

1. State is a built-in React object that holds data local to a component and determines how that component renders and behaves.

2. Unlike props, state is mutable and is managed within the component itself; changes to state trigger a re-render of the component and its children.

3. In functional components, state is managed via the useState Hook, while class components use this.state and this.setState().

Q7. What are React Hooks?

1. React Hooks are functions introduced in React 16.8 that allow functional components to use state and other React features without writing class components.

2. Core hooks include useState for local state, useEffect for side effects, useContext for context consumption, useRef for mutable references, and useMemo/useCallback for performance optimisation.

3. Hooks must be called at the top level of a functional component (not inside loops or conditionals) to ensure consistent hook call order across renders.

Q8. What is the useState Hook?

1. useState is a React Hook that adds local state to a functional component; it returns a state variable and a setter function as a destructured array.

2. The setter function triggers a re-render of the component with the new state value; React batches multiple state updates for efficiency.

3. Initial state can be a primitive, object, or array; for expensive initialisations, a lazy initialiser function can be passed to useState to run only on the first render.

Q9. What is the useEffect Hook?

1. useEffect is a React Hook that lets functional components perform side effects such as data fetching, subscriptions, timers, and manual DOM manipulation.

2. It accepts a callback function and an optional dependency array; the effect re-runs only when the specified dependencies change, replacing componentDidMount, componentDidUpdate, and componentWillUnmount lifecycle methods.

3. A cleanup function returned from useEffect is executed before the component unmounts or before the effect runs again, preventing memory leaks from subscriptions or event listeners.

Q10. What is the useContext Hook?

1. useContext is a React Hook that provides a way to consume a React Context value without wrapping components in a Context.Consumer component.

2. It accepts a context object (created by React.createContext) and returns the current context value, allowing deeply nested components to access shared data without prop drilling.

3. When the context value changes, all components consuming that context via useContext will automatically re-render with the updated value.

Q11. What is useRef in React?

1. useRef returns a mutable ref object whose .current property persists for the full lifetime of the component without triggering re-renders when changed.

2. It is commonly used to directly access and manipulate DOM elements (e.g., focusing an input) and to store mutable values like previous state or timer IDs.

3. Unlike state, mutating ref.current does not schedule a re-render, making it ideal for storing values that need to persist between renders but do not affect the UI.

Q12. What is useMemo in React?

1. useMemo is a React Hook that memoises the result of an expensive computation and returns the cached value unless its dependencies change.

2. It accepts a factory function and a dependency array; React skips re-running the function and returns the memoised value when dependencies are unchanged between renders.

3. useMemo optimises performance by preventing redundant expensive calculations on every render, but should only be used when profiling confirms a genuine performance bottleneck.

Q13. What is useCallback in React?

1. useCallback is a React Hook that returns a memoised version of a callback function, recreating it only when its dependencies change.

2. It is particularly useful when passing callbacks as props to child components wrapped in React.memo, preventing unnecessary re-renders caused by new function references on every parent render.

3. Like useMemo, useCallback should be applied judiciously—premature use can add overhead without meaningful performance gains.

Q14. What is the Context API?

1. The React Context API provides a mechanism to share values (such as themes, authentication, or locale) across the component tree without explicitly passing props at every level.

2. A context is created with React.createContext(), a Provider component wraps the subtree supplying the value, and consumers access it via useContext() or Context.Consumer.

3. Context is best suited for low-frequency updates (e.g., theme toggling); for high-frequency state management needs, dedicated libraries like Redux or Zustand are more performant.

Q15. What is React.memo?

1. React.memo is a higher-order component (HOC) that wraps a functional component and memoises its rendered output, preventing re-renders when props have not changed.

2. By default it performs a shallow comparison of props; a custom comparison function can be passed as the second argument for deeper equality checks.

3. It is most effective for pure presentational components that receive stable props, reducing unnecessary rendering overhead in large component trees.

Q16. What is the difference between controlled and uncontrolled components?

1. A controlled component has its form input value driven entirely by React state; every change fires an onChange handler that updates state, making React the single source of truth.

2. An uncontrolled component stores its own value in the DOM, accessed imperatively via a ref; it requires less boilerplate but offers less control over input validation and formatting.

3. Controlled components are preferred for complex forms requiring validation, conditional fields, or dynamic submission logic, while uncontrolled components suit simple, quick integrations.

Q17. What is React Router?

1. React Router is the de facto client-side routing library for React applications, enabling navigation between views without full page reloads.

2. It provides components such as BrowserRouter, Routes, Route, Link, and NavLink to declaratively define application routes and render matching components.

3. React Router v6 introduced relative routes, nested route layouts, and the useNavigate hook, significantly simplifying complex routing patterns compared to earlier versions.

Q18. What is Redux?

1. Redux is a predictable state management library based on the Flux architecture pattern, maintaining a single immutable state tree (store) for the entire application.

2. State changes are triggered exclusively by dispatching plain action objects to pure reducer functions, ensuring changes are traceable, testable, and reproducible.

3. Redux Toolkit (RTK) is the official, opinionated toolset that simplifies Redux usage by providing createSlice, createAsyncThunk, and built-in Immer integration for immutable updates.

Q19. What is prop drilling and how can it be avoided?

1. Prop drilling occurs when props must be passed through multiple intermediate components that do not use them, solely to reach a deeply nested child component.

2. It makes the codebase harder to maintain because intermediate components become unnecessarily coupled to data they do not consume.

3. Solutions include the React Context API for shared global data, state management libraries like Redux or Zustand, or component composition patterns to restructure the component hierarchy.

Q20. What are Higher-Order Components (HOCs)?

1. A Higher-Order Component is a function that takes a component as an argument and returns a new enhanced component, following the compositional pattern of higher-order functions.

2. HOCs are used to abstract cross-cutting concerns such as authentication guards, logging, data fetching, or theme injection without modifying the original component.

3. With the introduction of Hooks, many HOC use cases have been replaced by custom hooks, which are simpler, more composable, and do not create additional component tree nesting.

Q21. What are React lifecycle methods?

1. React class component lifecycle methods are hooks invoked at specific phases: mounting (constructor, componentDidMount), updating (shouldComponentUpdate, componentDidUpdate), and unmounting (componentWillUnmount).

2. componentDidMount is ideal for initiating API calls and subscriptions; componentDidUpdate handles side effects in response to prop/state changes; componentWillUnmount performs cleanup.

3. In functional components, useEffect with appropriate dependency arrays replicates all lifecycle behaviour, providing a unified API for side effect management.

Q22. What is React.lazy and Suspense?

1. React.lazy enables code-splitting by dynamically importing a component only when it is first rendered, reducing the initial JavaScript bundle size.

2. The lazily loaded component must be wrapped in a React.Suspense boundary that provides a fallback UI (e.g., a spinner) displayed while the component chunk is being fetched.

3. Code-splitting with React.lazy and Suspense is an effective performance optimisation for large applications, improving Time-to-Interactive by deferring non-critical code loading.

Q23. What is reconciliation in React?

1. Reconciliation is the process by which React determines the minimal set of DOM updates required when a component's state or props change.

2. React's diffing algorithm assumes elements of different types produce different trees and uses the key prop to track list items across renders, enabling O(n) complexity diffing.

3. Understanding reconciliation helps developers write performant components by avoiding unnecessary re-renders and correctly using keys in dynamic lists.

Q24. What are keys in React lists?

1. Keys are special string attributes that help React identify which items in a list have changed, been added, or been removed during reconciliation.

2. Keys must be unique among sibling elements and should be stable identifiers (e.g., database IDs) rather than array indices, which can cause bugs when items are reordered or deleted.

3. Providing correct keys enables React to efficiently update list items with minimal DOM mutations instead of re-rendering the entire list.

Q25. What is the difference between useEffect and useLayoutEffect?

1. useEffect runs asynchronously after the browser has painted the screen, making it suitable for side effects that do not require visual synchronisation (e.g., data fetching, subscriptions).

2. useLayoutEffect runs synchronously after all DOM mutations but before the browser paints, allowing DOM measurements and synchronous style/layout updates to prevent flicker.

3. useLayoutEffect should be used sparingly because blocking the paint thread can degrade perceived performance; prefer useEffect unless DOM measurement or synchronous update is required.

Q26. What is React's StrictMode?

1. React.StrictMode is a development-only tool that wraps a component tree and activates additional checks and warnings to help identify potential problems.

2. It intentionally double-invokes functions like render and state initialisers to detect side effects in these phases that should be pure, and warns about deprecated lifecycle usage.

3. StrictMode has no impact on production builds and is strongly recommended during development to surface future-compatibility issues early.

Q27. What is the difference between class and functional components?

1. Class components extend React.Component, define a render() method, and manage state and lifecycle via this.state and lifecycle methods; they were the primary component type before React 16.8.

2. Functional components are plain JavaScript functions that accept props and return JSX; with Hooks, they can manage state and side effects, making class components largely unnecessary for new code.

3. Functional components are preferred today for their simplicity, smaller bundle size, easier testing, and better compatibility with React's concurrent features and future roadmap.

Q28. What is the useReducer Hook?

1. useReducer is a React Hook that manages complex state logic using a reducer function, similar to Redux but scoped to a single component or subtree.

2. It accepts a reducer (state, action) => newState and an initial state, returning the current state and a dispatch function for triggering state transitions.

3. useReducer is preferable over useState when state updates involve multiple sub-values or when the next state depends on the previous one in non-trivial ways.

Q29. What is Lifting State Up in React?

1. Lifting state up is a React pattern of moving shared state to the closest common ancestor component so that multiple child components can access and synchronise around the same data.

2. The ancestor holds the state and passes it down as props, along with callback functions that allow children to request state changes, maintaining a unidirectional data flow.

3. This pattern keeps state management predictable and centralised, avoiding state duplication and the synchronisation bugs that arise from maintaining state in multiple sibling components.

Q30. What are portals in React?

1. React Portals allow a child component to be rendered into a different DOM node than its parent, while still preserving React's event bubbling and context hierarchy.

2. They are created using ReactDOM.createPortal(child, container) and are ideal for UI elements that must visually escape their container, such as modals, tooltips, and dropdown menus.

3. Despite rendering outside the parent DOM hierarchy, portal components still participate in the React component tree, meaning context and event propagation work as expected.

Q31. What is the purpose of the key prop in React?

1. The key prop is a special React attribute that uniquely identifies elements within a list, enabling React's reconciliation algorithm to efficiently track which items have changed.

2. Using stable, unique keys (such as database IDs) rather than array indices prevents subtle rendering bugs when items are inserted, deleted, or reordered.

3. Keys are not accessible as props in the component; if the key value is needed inside a component, it should be passed explicitly as a separate prop.

Q32. What is Concurrent Mode in React?

1. Concurrent Mode (now called Concurrent Features in React 18) allows React to interrupt, pause, resume, or abandon in-progress renders to keep the UI responsive during heavy computations.

2. It enables features like startTransition (marking updates as non-urgent), useDeferredValue (deferring expensive re-renders), and Suspense for data fetching.

3. Concurrent features improve user experience in large applications by ensuring high-priority updates (e.g., user input) are processed before lower-priority updates (e.g., data renders).

Q33. What is error boundary in React?

1. An error boundary is a React class component that implements componentDidCatch and/or getDerivedStateFromError to catch JavaScript errors in its child component tree during rendering.

2. When an error is caught, the boundary can render a fallback UI instead of crashing the entire application, improving resilience and user experience.

3. Error boundaries do not catch errors in event handlers, asynchronous code, or SSR; those must be handled with traditional try/catch blocks.

Q34. What is the difference between createElement and JSX?

1. JSX is syntactic sugar for React.createElement(type, props, ...children) calls; Babel transforms JSX into these function calls before code is executed by the browser.

2. React.createElement can be used directly when a build step is not available or when generating elements programmatically, but it is far more verbose than JSX for nested structures.

3. Both produce React element objects (plain JavaScript objects describing the UI), which React then uses during reconciliation to update the DOM efficiently.

Q35. What is a custom Hook in React?

1. A custom Hook is a JavaScript function whose name starts with 'use' that encapsulates and reuses stateful logic by calling built-in React Hooks internally.

2. Custom Hooks allow cross-component logic sharing (e.g., useFetch, useDebounce, useLocalStorage) without introducing extra component tree nesting like HOCs or render props.

3. Each call to a custom Hook has its own isolated state, ensuring that logic is shared but state is not, maintaining React's composability principles.

Q36. What is React.forwardRef?

1. React.forwardRef is a utility that allows a parent component to pass a ref through a functional component to one of its DOM children or inner components.

2. Without forwardRef, refs cannot be attached to functional components because they do not have instances; forwardRef exposes the inner DOM node via the ref prop.

3. It is commonly used in reusable UI libraries to give consuming components direct access to underlying DOM elements for focus management, animations, or scroll control.

Q37. What is the significance of the children prop?

1. The children prop is a special React prop that contains the nested JSX or components placed between a component's opening and closing tags.

2. It enables component composition, allowing a parent component to act as a wrapper or layout container without knowing the specific content it will render.

3. React.Children utilities (map, forEach, count) provide methods for safely iterating over and manipulating the children prop, handling edge cases like undefined or single children.

Q38. What is code splitting in React?

1. Code splitting is a build optimisation technique that divides the JavaScript bundle into smaller chunks that are loaded on demand rather than all at once on initial page load.

2. React supports code splitting natively via dynamic import() syntax combined with React.lazy and Suspense, enabling route-level or component-level splitting.

3. Reducing the initial bundle size with code splitting improves First Contentful Paint (FCP) and Time-to-Interactive (TTI), particularly for large single-page applications.

Q39. What are render props in React?

1. The render props pattern is a technique where a component receives a function as a prop and calls it to determine what to render, sharing logic without inheritance.

2. This allows a component to expose its internal state or behaviour to a parent or sibling, enabling flexible reuse of stateful logic across different rendering contexts.

3. Like HOCs, render props have largely been superseded by custom Hooks, which achieve the same logic sharing in a more concise and composable manner.

Q40. What is React's synthetic event system?

1. React implements a synthetic event system that wraps native browser events in a cross-browser compatible SyntheticEvent object, normalising differences between browsers.

2. Synthetic events are pooled for performance in older React versions; in React 17+, event delegation was moved from the document to the React root element, improving interoperability.

3. Developers interact with synthetic events the same way as native events, using event handlers like onClick, onChange, and onSubmit, while React handles cross-browser compatibility internally.

Q41. What is the difference between shallow and deep rendering in React testing?

1. Shallow rendering (via Enzyme's shallow or React Testing Library with mocks) renders a component one level deep without rendering child components, enabling isolated unit testing.

2. Deep/full rendering renders the complete component tree including all children, providing a more realistic test environment that captures interaction between components.

3. React Testing Library encourages full rendering with user-centric queries, while shallow rendering is useful for pure unit tests focused on a single component's logic and output.

Q42. What is React Testing Library?

1. React Testing Library (RTL) is a lightweight testing utility built on top of DOM Testing Library that encourages testing components as users interact with them rather than testing implementation details.

2. RTL provides queries like getByRole, getByText, and getByLabelText that mirror how users and assistive technologies navigate the DOM, promoting accessible component design.

3. By avoiding testing internal state and rendering details, RTL tests are more resilient to refactoring, reducing false positives and negatives compared to implementation-focused tests.

Q43. What is Flux architecture?

1. Flux is an application architecture pattern introduced by Facebook that enforces a strict unidirectional data flow: Actions → Dispatcher → Stores → Views.

2. Stores hold the application state and logic; when an action is dispatched, all relevant stores update their state and notify registered view components to re-render.

3. Redux is the most popular implementation of the Flux pattern, simplifying it by consolidating multiple stores into a single store and replacing the Dispatcher with pure reducer functions.

Q44. What is server-side rendering (SSR) in React?

1. Server-side rendering generates the initial HTML of a React application on the server and sends a fully rendered page to the client, improving First Contentful Paint and SEO.

2. Frameworks like Next.js provide built-in SSR support via getServerSideProps, which fetches data at request time and passes it to the page component before rendering.

3. SSR adds server infrastructure complexity and increases Time-to-First-Byte compared to client-side rendering, making it a trade-off suited for content-heavy, SEO-critical applications.

Q45. What is static site generation (SSG) in React?

1. Static Site Generation pre-renders pages at build time, producing static HTML files that can be served directly from a CDN without server computation per request.

2. Next.js supports SSG via getStaticProps and getStaticPaths, allowing dynamic pages to be pre-generated at build time with the data they need.

3. SSG provides the best performance and scalability for content that does not change frequently, such as documentation, marketing pages, or blogs, while reducing server costs.

Q46. What is the difference between SSR and CSR?

1. Client-Side Rendering (CSR) sends a minimal HTML shell and a JavaScript bundle; the browser executes the JS to fetch data and render the UI, resulting in a slower initial load but faster subsequent navigation.

2. Server-Side Rendering (SSR) generates the full HTML on the server per request, enabling faster initial page display and better SEO at the cost of higher server load and TTFB.

3. Modern frameworks like Next.js support hybrid approaches, allowing per-page decisions to use SSR, SSG, or CSR based on the specific requirements of each route.

Q47. What is React's batching mechanism?

1. React batches multiple state updates that occur within the same synchronous event handler into a single re-render, reducing unnecessary renders and improving performance.

2. In React 18, automatic batching was extended to asynchronous operations (Promises, setTimeout, native event handlers), which previously triggered individual re-renders for each setState call.

3. Developers can opt out of batching for a specific update using ReactDOM.flushSync(), which forces synchronous re-rendering immediately after the enclosed state update.

Q48. What is the useImperativeHandle Hook?

1. useImperativeHandle is a React Hook used with forwardRef to customise the instance value (ref handle) that is exposed to parent components when they attach a ref to the child.

2. Instead of exposing the entire DOM node, it allows the child component to expose only specific methods or values, encapsulating implementation details and providing a controlled API.

3. It is commonly used in reusable input or modal components where the parent needs to trigger specific actions (e.g., focus, reset, scroll) without accessing the full DOM element.

Q49. What is the purpose of getDerivedStateFromProps?

1. getDerivedStateFromProps is a static class component lifecycle method that returns an object to update state based on incoming props, or null if no update is needed.

2. It is called before every render (both on mount and on updates), replacing the deprecated componentWillReceiveProps with a safer, side-effect-free alternative.

3. This method should be used sparingly; most use cases it was intended for can be handled more cleanly by deriving values directly during render or using memoisation.

Q50. What is React's reconciliation diffing algorithm?

1. React's diffing algorithm operates on two assumptions: elements of different types produce completely different trees, and the developer can hint at stable child identity using the key prop.

2. When comparing two trees, React first checks the root element type; different types cause a full subtree teardown and rebuild, while same types result in attribute updates only.

3. For lists, React uses keys to match old and new elements, enabling it to reorder, insert, and remove items efficiently rather than re-rendering the entire list.

Q51. What is Next.js?

1. Next.js is a React-based full-stack framework by Vercel that provides built-in support for SSR, SSG, Incremental Static Regeneration (ISR), API routes, file-based routing, and image optimisation.

2. It simplifies production React development by eliminating the need to manually configure Webpack, Babel, and routing, providing an opinionated but highly flexible developer experience.

3. Next.js 13+ introduced the App Router with React Server Components, co-located layouts, and streaming, representing a significant shift toward server-centric React application architecture.

Q52. What are React Server Components?

1. React Server Components (RSCs) are components that render exclusively on the server and never ship their JavaScript to the client, reducing bundle size and enabling direct server-side data access.

2. RSCs can fetch data directly from databases or APIs without exposing credentials to the client, and they interleave seamlessly with Client Components in the same component tree.

3. RSCs represent a new rendering paradigm that blurs the boundary between server and client, enabling more efficient data loading patterns and reducing client-side JavaScript overhead.

Q53. What is Incremental Static Regeneration (ISR)?

1. ISR is a Next.js feature that allows statically generated pages to be updated after build time on a per-page basis by specifying a revalidation interval in getStaticProps.

2. When a request comes in after the revalidation period, Next.js serves the stale cached page immediately and regenerates a fresh version in the background for subsequent requests.

3. ISR combines the performance of static generation with the data freshness of server-side rendering, making it ideal for pages with frequently but not continuously changing content.

Q54. What is Zustand?

1. Zustand is a minimalist, hook-based state management library for React that provides a global store without the boilerplate of Redux.

2. It uses a simple create function to define a store with state and actions; components subscribe to the store via a hook and automatically re-render only when their selected state slice changes.

3. Zustand's lack of providers, reducers, and actions makes it significantly simpler than Redux for small-to-medium applications while still supporting middleware and devtools integration.

Q55. What is React Query (TanStack Query)?

1. React Query (now TanStack Query) is a data-fetching and server-state synchronisation library that manages caching, background refetching, deduplication, and loading/error states automatically.

2. It uses a query key system to uniquely identify and cache server data; stale data is automatically refetched in the background based on configurable stale time and refetch policies.

3. React Query significantly reduces client-side state management complexity by treating server state as a distinct concern separate from UI/client state, eliminating the need for manual cache management in Redux or Context.

Q56. What is the difference between React and React Native?

1. React is a JavaScript library for building web user interfaces that renders to the browser DOM using HTML elements such as div, span, and input.

2. React Native is a framework for building native mobile applications (iOS and Android) using React's component model, rendering to native platform UI components instead of the DOM.

3. Both share the same core React paradigm (components, props, state, hooks), allowing developers with React expertise to transfer their knowledge to mobile development with React Native.

Q57. What is Vite and how does it differ from Create React App?

1. Vite is a next-generation frontend build tool that uses native ES modules in the browser during development, providing near-instant server start and Hot Module Replacement (HMR).

2. Create React App (CRA) bundles the entire application on startup using Webpack, resulting in slow cold starts for large projects; Vite avoids this by serving unbundled source files and bundling on demand.

3. Vite has become the community-preferred scaffolding tool for new React projects due to its dramatically faster development experience, while CRA is officially in maintenance mode.

Q58. What are React fragments?

1. React Fragments (or the shorthand <>) allow a component to return multiple sibling elements without adding an extra DOM node as a wrapper.

2. They are useful when inserting elements into lists or tables where wrapping divs would produce invalid HTML or interfere with CSS layout (e.g., Flexbox/Grid children).

3. The shorthand syntax <> does not support the key prop; use when rendering keyed lists of fragments.

Q59. What is the difference between imperative and declarative programming in React context?

1. Imperative programming describes the step-by-step commands to achieve a result (e.g., directly manipulating the DOM: document.getElementById().style.color = 'red').

2. Declarative programming describes what the UI should look like for a given state, and React handles the DOM updates internally: .

3. React's declarative model abstracts away DOM manipulation, making code more predictable, readable, and easier to reason about, especially as application complexity grows.

Q60. What is the significance of React 18's startTransition API?

1. startTransition marks a state update as a non-urgent transition, allowing React to defer re-rendering it in favour of more urgent updates like user input, keeping the UI responsive.

2. It addresses the challenge of large, slow renders (e.g., filtering a long list) blocking user interactions by separating urgent from non-urgent state updates at the API level.

3. The useTransition hook exposes startTransition along with an isPending flag, enabling UI feedback (e.g., a loading indicator) while the transition is in progress.

Q61. What is prop types validation in React?

1. PropTypes is a runtime type-checking library for React props that logs console warnings during development when a component receives props of an unexpected type.

2. It is defined as a static propTypes property on a component, specifying the expected type, shape, and required status of each prop using PropTypes validators.

3. For static type safety, TypeScript interfaces or types are preferred over PropTypes in production codebases, as they catch type errors at compile time rather than runtime.

Q62. What is the component composition pattern in React?

1. Component composition is the practice of building complex UIs by combining smaller, focused components rather than creating monolithic components with many responsibilities.

2. React's children prop and the concept of 'slots' allow container components to accept and render arbitrary content, enabling highly flexible and reusable layouts.

3. Composition is preferred over inheritance for code reuse in React; the React documentation explicitly recommends it as the primary model for component reuse and extension.

Q63. What is the difference between React.Component and React.PureComponent?

1. React.Component re-renders whenever setState is called or new props are received, regardless of whether the actual values have changed.

2. React.PureComponent implements shouldComponentUpdate with a shallow comparison of props and state, preventing re-renders when neither has changed, improving performance.

3. In functional components, React.memo provides equivalent behaviour to PureComponent; for deep object comparisons, a custom areEqual function should be provided to avoid false positives.

Q64. What is the Flux unidirectional data flow?

1. Unidirectional data flow means data travels in a single direction: from parent components to child components via props, and state changes bubble up via callback props.

2. This makes the application state predictable and traceable since there is a clear, linear path from state to rendered UI, simplifying debugging and reasoning about component behaviour.

3. React's one-way binding contrasts with two-way binding frameworks (e.g., Angular's ngModel), trading some syntactic convenience for greater clarity and fewer data synchronisation bugs.

Q65. What is the purpose of the displayName property in React?

1. The displayName property is a string that identifies a component in React DevTools, error messages, and stack traces, making debugging significantly easier.

2. For HOCs and components created dynamically, React cannot always infer a meaningful name from the function; setting displayName provides a human-readable label explicitly.

3. It is particularly useful in production debugging and when building component libraries where clear component identification in DevTools improves developer experience.

Q66. What are the rules of Hooks?

1. React Hooks must only be called at the top level of a functional component or custom Hook—never inside loops, conditions, or nested functions—to ensure hook call order is consistent across renders.

2. Hooks must only be called from React functional components or custom Hooks, not from regular JavaScript functions, class components, or event handlers outside components.

3. The eslint-plugin-react-hooks package enforces these rules statically at development time, preventing subtle bugs caused by conditional or out-of-order hook calls.

Q67. What is the purpose of React DevTools?

1. React DevTools is a browser extension and standalone tool that lets developers inspect the React component tree, view props and state for each component, and monitor context values.

2. The Profiler tab records rendering performance, showing which components re-rendered, how long each render took, and which hook or prop change triggered the re-render.

3. React DevTools is essential for debugging complex state issues, identifying performance bottlenecks, and understanding component hierarchy in large applications.

Q68. What is Tailwind CSS and how is it used with React?

1. Tailwind CSS is a utility-first CSS framework that provides a comprehensive set of pre-built CSS utility classes applied directly in JSX className attributes.

2. Instead of writing custom CSS or using CSS-in-JS, developers compose designs directly in markup using classes like flex, text-lg, bg-blue-500, and rounded-md.

3. Tailwind integrates with React projects via PostCSS configuration and tree-shakes unused styles at build time, producing minimal CSS bundles while enabling rapid UI development.

Q69. What are CSS Modules in React?

1. CSS Modules are CSS files where class names are automatically scoped locally to the component that imports them, preventing style collisions in large applications.

2. Webpack or Vite process CSS Module files (e.g., Button.module.css) and generate unique hashed class names, which are imported as a JavaScript object and applied via className={styles.button}.

3. CSS Modules combine the familiarity of standard CSS syntax with local scoping, providing a good balance between simplicity and encapsulation without requiring a CSS-in-JS runtime.

Q70. What is Styled Components?

1. Styled Components is a CSS-in-JS library that allows writing actual CSS syntax inside JavaScript template literals to create styled React components.

2. Styles are scoped to the component, support dynamic styling via props, and are automatically injected into the document, eliminating the need for separate CSS files.

3. The library handles vendor prefixing, theming via a ThemeProvider, and server-side rendering style injection, providing a powerful styling solution for React component libraries.

Q71. What is the difference between React and Angular?

1. React is a UI library focused solely on the view layer, requiring developers to choose their own solutions for routing, state management, and HTTP; Angular is a comprehensive MVC framework that provides all these out of the box.

2. React uses a Virtual DOM and JSX for rendering; Angular uses a real DOM with change detection via zones and an HTML-based template syntax with directives.

3. React has a smaller learning curve for JavaScript developers familiar with functional programming; Angular has a steeper curve due to TypeScript requirements, decorators, and its opinionated structure.

Q72. What is the difference between React and Vue?

1. React is a library with minimal opinions, giving developers maximum flexibility in choosing tooling, patterns, and architecture; Vue is a progressive framework with more built-in opinions and a gentler learning curve.

2. Vue uses an HTML template syntax with two-way data binding (v-model) and a single-file component format (.vue); React uses JSX and enforces unidirectional data flow.

3. Both leverage Virtual DOM and component-based architecture; React has a larger ecosystem and corporate backing from Meta, while Vue has a dedicated open-source community and is favoured in Asia-Pacific markets.

Q73. What is the significance of the key prop when rendering dynamic lists?

1. Without stable keys, React falls back to index-based reconciliation, causing components to retain stale state when items are reordered, inserted, or deleted.

2. Using stable unique identifiers as keys (e.g., item.id) ensures React correctly matches old and new list items during reconciliation, preserving component state accurately.

3. Incorrect key usage is a common source of subtle bugs in React applications, particularly in forms or animated lists where component state must persist across dynamic updates.

Q74. What is the React event delegation model?

1. React attaches a single event listener at the root DOM element (React 17+) rather than individual DOM nodes, using event delegation to handle all events centrally.

2. When a native event fires, it bubbles up to the React root, which dispatches the corresponding synthetic event to the appropriate React component handler.

3. This model reduces memory consumption by avoiding per-element listeners and enables React to manage event normalisation and cleanup consistently across the entire application.

Q75. What is the purpose of useDebugValue in React?

1. useDebugValue is a Hook that adds a label to custom Hooks in React DevTools, making it easier to identify and inspect custom hook state during debugging.

2. It accepts a value to display and an optional format function that is called lazily, only when the DevTools panel is open, to avoid expensive formatting in production.

3. It is a developer experience utility with no effect on application behaviour and is particularly valuable in shared custom Hook libraries to provide descriptive labels in DevTools.

Q76. What is Formik and how is it used in React?

1. Formik is a popular form management library for React that handles form state, validation, error messages, and submission, reducing boilerplate in complex forms.

2. It provides the useFormik hook and Form, Field, and ErrorMessage components; validation can be defined inline or via Yup schema validation for declarative rules.

3. Formik integrates seamlessly with UI libraries and custom inputs via the Field component's render prop pattern, enabling consistent form behaviour across large applications.

Q77. What is React's hydration process?

1. Hydration is the process of attaching React's event system and making a server-rendered HTML page interactive on the client without re-rendering the entire DOM from scratch.

2. React compares the server-rendered HTML with the client-side Virtual DOM; if they match, React attaches event handlers to existing DOM nodes rather than replacing them.

3. Hydration mismatches (differences between server and client render) cause React to warn in development and can degrade performance; they must be carefully avoided in SSR applications.

Q78. What are micro-frontends and how do they relate to React?

1. Micro-frontends is an architectural pattern that decomposes a frontend application into independently deployable vertical slices owned by different teams.

2. React applications can serve as micro-frontends by being composed into a shell application using Module Federation (Webpack 5), iframes, or web components.

3. While micro-frontends improve team autonomy and independent deployability, they introduce complexity around shared dependencies, consistent styling, and cross-app communication that must be carefully managed.

Q79. What is the significance of React 16's Fiber architecture?

1. React Fiber is a complete rewrite of React's core reconciliation algorithm introduced in React 16 to enable asynchronous and interruptible rendering.

2. Fiber represents each unit of work as a linked list of nodes (fibres), allowing React to pause, prioritise, and resume rendering work based on priority levels rather than processing the tree synchronously.

3. Fiber laid the groundwork for all concurrent features in React 18 (startTransition, Suspense for data, useTransition) by making the rendering pipeline pausable and priority-aware.

Q80. What is the difference between eager and lazy state initialisation in useState?

1. Passing a value directly to useState (eager initialisation) evaluates the expression on every render, even though React only uses the initial value on the first render.

2. Passing a function to useState (lazy initialisation) means React calls the function only on the first render, avoiding potentially expensive computations on subsequent re-renders.

3. Lazy initialisation is important when the initial state requires reading from localStorage, performing complex calculations, or processing large data structures that would degrade performance if re-computed on every render.

Q81. What is the purpose of the React.Children API?

1. React.Children is a utility object that provides methods (map, forEach, count, only, toArray) for safely working with the children prop, handling edge cases like null, undefined, or single children.

2. React.Children.map applies a function to each child and returns a new array, while React.Children.toArray flattens and normalises children into a flat array with stable keys.

3. Direct array manipulation of children is discouraged because children can be a single element, an array, or undefined; the React.Children API handles all cases consistently.

Q82. What is the stale closure problem in React Hooks?

1. The stale closure problem occurs when a Hook's callback captures an outdated value of a state or prop variable from its enclosing scope due to JavaScript's lexical scoping rules.

2. This commonly manifests in useEffect, useCallback, or event handlers that reference state variables not included in their dependency array, causing them to operate on stale data.

3. Solutions include adding all referenced variables to the dependency array, using the functional update form of setState (prev => prev + 1), or using useRef to store mutable values that don't trigger re-renders.

Q83. What is the React Context pitfall regarding re-renders?

1. All components that consume a React Context will re-render whenever the context value changes, regardless of whether the specific part of the value they use has changed.

2. Passing large objects as context values causes broad re-renders across the consumer tree even for minor property changes; splitting context into multiple focused providers mitigates this.

3. Memoising context values with useMemo and splitting the context into separate providers for different update frequencies are common optimisation strategies for performance-sensitive context usage.

Q84. What is React's `act()` utility in testing?

1. act() is a testing utility that wraps code that causes React state updates, ensuring all pending state updates, effects, and re-renders are processed before assertions are made.

2. Without act(), tests may make assertions on intermediate UI states before React has finished processing updates, leading to false failures or unreliable test results.

3. React Testing Library wraps its user interaction utilities (userEvent, fireEvent) in act() automatically, but direct state updates or async operations in tests often require explicit act() wrapping.

Q85. What is tree shaking in the context of React applications?

1. Tree shaking is a dead code elimination optimisation performed by bundlers like Webpack and Rollup that removes unused exports from the final bundle.

2. React's modular architecture and ES module exports allow bundlers to tree-shake unused components, hooks, and utilities, reducing bundle size significantly for large applications.

3. Ensuring libraries are tree-shakeable (using ES module format, avoiding side effects in module scope) and importing only needed functions (import { useState } rather than import React) maximises tree shaking effectiveness.

Q86. What is the purpose of React's defaultProps?

1. defaultProps is a static property that defines default values for a component's props, applied when those props are undefined (but not when they are null).

2. For class components, defaultProps is defined as a static property; for functional components, default parameter values in the function signature are the modern equivalent.

3. TypeScript users typically express defaults via default parameter values in conjunction with interface definitions rather than defaultProps, as it integrates more naturally with the type system.

Q87. What is the difference between mount and render in React testing?

1. In Enzyme terminology, mount performs a full DOM render of the component tree using a simulated DOM environment, enabling testing of lifecycle methods and child component interactions.

2. shallow renders only the component under test without its children, providing fast, isolated unit tests that don't require a full DOM environment.

3. In React Testing Library, render is the primary API and always performs a full DOM render into a real jsdom environment, reflecting the library's philosophy of testing like a real user.

Q88. What is the useId Hook introduced in React 18?

1. useId is a React 18 Hook that generates stable, unique IDs that are consistent between server and client renders, specifically designed for accessibility attributes like aria-labelledby and htmlFor.

2. Unlike Math.random() or incrementing counters, useId produces the same ID on both server and client, preventing hydration mismatches in SSR applications.

3. Each call to useId within a component produces a unique ID, and suffix variations (id + '-name', id + '-email') allow a single useId call to generate multiple related IDs.

Q89. What is the Compound Component pattern in React?

1. The Compound Component pattern models a group of closely related components that share implicit state via React Context, providing a flexible, expressive API for complex UI components.

2. A parent component manages shared state and exposes it via context; child components (compound components) consume that context, allowing consumers to compose them in any order.

3. Common examples include Tab/Tabs/TabPanel groups, Select/Option groups, and Accordion/AccordionItem groups, where the overall behaviour depends on the coordinated state of all parts.

Q90. What is the purpose of React's experimental use() API?

1. The use() API (experimental in React, stable in Next.js App Router) is a Hook that can be called inside render to read the value of a Promise or Context, suspending the component until the Promise resolves.

2. Unlike useEffect-based data fetching, use() integrates with Suspense natively, allowing async data to be read synchronously in the component body with React managing the loading state.

3. use() represents a fundamental shift in React's data fetching model, enabling server components to fetch data directly and client components to consume Promises as first-class values.

Q91. What is the difference between onBlur and onChange in React forms?

1. onChange fires on every keystroke or value change in an input element, making it suitable for real-time validation, character counting, or dependent field updates.

2. onBlur fires when the input loses focus, commonly used to trigger validation after the user has finished interacting with a field to avoid premature error messages.

3. Most form libraries (Formik, React Hook Form) use a combination of both: onChange for value tracking and onBlur for validation triggering to provide a good user experience.

Q92. What is React Hook Form?

1. React Hook Form is a performant form library that minimises re-renders by using uncontrolled inputs with ref-based value tracking rather than controlled state management.

2. It provides a useForm hook that returns register, handleSubmit, and formState; inputs are registered via the register function, which attaches the necessary refs and validation rules.

3. React Hook Form is significantly more performant than Formik for large forms because it avoids re-rendering the entire form on every input change, updating only the fields with errors.

Q93. What is the purpose of StrictMode double-invocation?

1. In React 18 development mode, StrictMode intentionally double-invokes component functions, state initialisers, and certain lifecycle methods to detect impure code that relies on side effects during rendering.

2. Since rendering should be a pure function of state and props, any observable difference between the two invocations indicates a bug where side effects are incorrectly placed in the render phase.

3. Only the first invocation's output is used; the double call is invisible to users in production but helps developers catch bugs where render functions mutate external state, perform async operations, or access non-deterministic values.

Q94. What is the Suspense component used for in React?

1. React.Suspense wraps components that may need to 'suspend' (pause rendering) while waiting for asynchronous data or code, displaying a fallback UI during the wait.

2. It works with React.lazy for code-splitting and with data-fetching solutions that support the Suspense protocol (like React Query's experimental Suspense mode and use() in RSCs).

3. Suspense enables declarative loading states without manual isLoading flags, making loading UI a first-class concern in the component tree and simplifying async rendering patterns.

Q95. What is the React Profiler API?

1. The React Profiler API (component and ReactDOM.unstable_Profiler) measures the rendering cost of a component tree by recording commit duration, phase, and interaction counts.

2. The onRender callback receives information about each committed render, including actualDuration (time spent rendering), baseDuration (estimated time without memoisation), and which interactions triggered the render.

3. Profiler is disabled in production builds; in development, it should be used in conjunction with React DevTools' Profiler tab to identify and resolve performance bottlenecks.

Q96. What is the purpose of the getDerivedStateFromError lifecycle method?

1. getDerivedStateFromError is a static class component lifecycle method called when a descendant component throws an error during rendering, lifecycle methods, or constructors.

2. It receives the thrown error and returns a state update object to set the error boundary's state, which then controls the rendering of a fallback UI instead of the crashed subtree.

3. It is called during the render phase so it must be side-effect free; componentDidCatch is used for logging error details to external monitoring services (e.g., Sentry).

Q97. What is the React reconciliation key heuristic?

1. React's reconciliation algorithm assumes that components at the same position in the tree with the same key represent the same component instance and should be updated rather than replaced.

2. Changing a component's key forces React to unmount the old instance and mount a completely new one, effectively resetting all local state; this is a deliberate pattern for forcing re-initialisation.

3. Misusing keys (e.g., using array index as key for reorderable lists) breaks the key heuristic and causes incorrect state retention, particularly in form inputs and animated components.

Q98. What is the difference between ReactDOM.render and ReactDOM.createRoot?

1. ReactDOM.render (legacy mode) is the React 17 and earlier API for mounting a React app into a DOM container; it renders synchronously and does not support concurrent features.

2. ReactDOM.createRoot (React 18+) creates a concurrent-capable root that supports all React 18 features including automatic batching, startTransition, and Suspense improvements.

3. Migration from legacy render to createRoot is required to unlock React 18 concurrent features; both APIs coexist in React 18 to support gradual migration in large codebases.

Q99. What is the purpose of the useTransition Hook?

1. useTransition is a React 18 Hook that returns [isPending, startTransition], allowing components to mark state updates as non-urgent transitions that can be deferred.

2. While the transition is pending, React keeps the current UI interactive and visible; isPending can be used to display a loading indicator without a full page spinner.

3. It is ideal for search-as-you-type interactions, tab switching with expensive renders, or any scenario where the new UI takes time to compute and the old UI should remain responsive.

Q100. What is the difference between React's controlled and uncontrolled form patterns?

1. Controlled forms use React state as the source of truth for input values; every change triggers an onChange handler updating state, providing full programmatic control over the input at every keystroke.

2. Uncontrolled forms let the DOM manage input values natively, accessed imperatively via refs only when needed (e.g., on submit), reducing re-renders and simplifying integration with non-React code.

3. Controlled forms are preferred for complex validation, dependent fields, and dynamic forms; uncontrolled forms are adequate for simple forms or when integrating with non-React libraries that manage their own input state.

SECTION 2: MySQL (Questions 101–200)

Q101. What is MySQL?

1. MySQL is an open-source relational database management system (RDBMS) that organises data into structured tables with rows and columns, using SQL (Structured Query Language) for data manipulation.

2. It uses a client-server architecture where the MySQL server manages databases and clients (applications, CLI tools) communicate with it over TCP/IP or Unix sockets.

3. MySQL supports ACID transactions (with InnoDB), replication, full-text search, JSON data types, and is widely used in web applications as the 'M' in the LAMP and MEAN stacks.

Q102. What are the main MySQL storage engines?

1. InnoDB is the default MySQL storage engine, supporting ACID transactions, foreign keys, row-level locking, and crash recovery via redo/undo logs, making it suitable for most production workloads.

2. MyISAM is a legacy engine that does not support transactions or foreign keys but offers faster full-text search and table-level locking; it is largely superseded by InnoDB in modern MySQL.

3. Memory (HEAP) stores all data in RAM for extremely fast access and is suitable for temporary tables and caching, but all data is lost on server restart.

Q103. What is a primary key in MySQL?

1. A primary key is a column or combination of columns that uniquely identifies each row in a table; MySQL enforces uniqueness and NOT NULL constraints automatically on primary key columns.

2. InnoDB physically orders table data on disk by the primary key (clustered index), making primary key lookups the fastest possible access pattern for row retrieval.

3. Best practice is to use a surrogate key (AUTO_INCREMENT integer or UUID) as the primary key rather than natural keys, which may change over time and complicate foreign key relationships.

Q104. What is a foreign key in MySQL?

1. A foreign key is a column (or set of columns) in a child table whose values reference the primary key of a parent table, enforcing referential integrity at the database level.

2. MySQL (InnoDB) validates foreign key constraints on INSERT, UPDATE, and DELETE operations, preventing orphaned records and data inconsistencies across related tables.

3. Foreign key constraints support ON DELETE and ON UPDATE actions (CASCADE, SET NULL, RESTRICT, NO ACTION) to define how child rows respond to changes in the referenced parent row.

Q105. What is normalisation in MySQL?

1. Database normalisation is the process of structuring relational tables to reduce data redundancy and improve data integrity by organising columns and tables according to normal forms.

2. 1NF requires atomic column values; 2NF eliminates partial dependencies on composite keys; 3NF eliminates transitive dependencies; BCNF, 4NF, and 5NF address more complex anomalies.

3. Normalisation reduces update anomalies and storage requirements but may require more JOINs in queries; denormalisation is sometimes applied strategically for read-performance optimisation.

Q106. What is an index in MySQL?

1. An index is a data structure (typically a B-Tree) maintained by MySQL alongside a table that enables fast row lookups based on specific column values, avoiding full table scans.

2. Indexes dramatically speed up SELECT queries with WHERE, JOIN, ORDER BY, and GROUP BY clauses but incur overhead on INSERT, UPDATE, and DELETE operations due to index maintenance.

3. MySQL supports various index types: PRIMARY KEY, UNIQUE, INDEX (non-unique), FULLTEXT (for text search), and SPATIAL (for geometric data), each optimised for different query patterns.

Q107. What is the difference between INNER JOIN and LEFT JOIN?

1. INNER JOIN returns only the rows where the join condition is satisfied in both tables, excluding rows from either table that have no matching row in the other.

2. LEFT JOIN (LEFT OUTER JOIN) returns all rows from the left table and matching rows from the right table; non-matching right table columns are filled with NULL for unmatched left rows.

3. Choosing the correct join type is critical for query correctness: INNER JOIN is used when both sides must have matching records; LEFT JOIN is used when all records from the driving table must be retained.

Q108. What is the difference between WHERE and HAVING clauses?

1. WHERE filters individual rows before grouping occurs, operating on raw column values and is processed before the GROUP BY clause.

2. HAVING filters groups after the GROUP BY clause has aggregated rows, allowing conditions on aggregate functions (SUM, COUNT, AVG) that cannot appear in the WHERE clause.

3. For performance, WHERE should be used whenever possible to reduce the number of rows before aggregation; HAVING is reserved for conditions that genuinely require aggregate results.

Q109. What are transactions in MySQL?

1. A transaction is a sequence of one or more SQL operations treated as a single atomic unit of work that either fully completes (COMMIT) or fully rolls back (ROLLBACK) on failure.

2. MySQL transactions follow the ACID properties: Atomicity (all-or-nothing), Consistency (transitions between valid states), Isolation (concurrent transactions appear serial), Durability (committed data persists).

3. Transactions are critical for operations that must maintain data integrity across multiple related tables, such as transferring funds between bank accounts or processing multi-item orders.

Q110. What are the ACID properties in MySQL?

1. Atomicity ensures that all operations within a transaction either fully succeed or are fully rolled back; there is no partial completion.

2. Consistency ensures that a transaction brings the database from one valid state to another, enforcing all defined constraints, rules, and cascades.

3. Isolation determines how concurrent transactions interact with each other; MySQL InnoDB supports isolation levels from READ UNCOMMITTED to SERIALIZABLE, controlling the trade-off between consistency and concurrency.

Q111. What are MySQL isolation levels?

1. READ UNCOMMITTED allows dirty reads (reading uncommitted changes from other transactions), providing the highest concurrency but the weakest consistency guarantees.

2. READ COMMITTED prevents dirty reads but allows non-repeatable reads; REPEATABLE READ (InnoDB default) prevents both dirty and non-repeatable reads using consistent snapshot reads.

3. SERIALIZABLE is the strictest level, executing transactions as if they were serial (one at a time), preventing all concurrency anomalies at the cost of significant performance reduction due to full locking.

Q112. What is a subquery in MySQL?

1. A subquery (inner query) is a SELECT statement nested within another SQL statement (outer query) that is evaluated first, with its result used by the outer query.

2. Subqueries can appear in SELECT, FROM, WHERE, and HAVING clauses; correlated subqueries reference columns from the outer query and are re-evaluated for each outer row.

3. For performance, JOINs are generally preferred over correlated subqueries because MySQL can optimise JOIN execution plans more effectively than repeatedly executing correlated subquery evaluations.

Q113. What is a stored procedure in MySQL?

1. A stored procedure is a named, precompiled block of SQL and procedural logic stored in the MySQL database that can be invoked by name with parameters.

2. Stored procedures reduce network round trips by executing complex multi-step logic on the server, support conditional logic (IF/CASE), loops (WHILE/LOOP), and variable manipulation via MySQL's

procedural language.

3. While stored procedures centralise business logic in the database and improve security via encapsulation, they complicate version control, testing, and application portability compared to application-layer logic.

Q114. What is a trigger in MySQL?

1. A trigger is a stored program that is automatically executed by MySQL in response to INSERT, UPDATE, or DELETE events on a specified table.

2. Triggers can be defined as BEFORE or AFTER the triggering event and access the old (OLD.*) and new (NEW.*) row values to enforce complex data integrity rules or audit logging.

3. While powerful, triggers execute invisibly from the application layer, making debugging difficult and potentially causing unexpected performance degradation; they should be used sparingly and documented thoroughly.

Q115. What is a view in MySQL?

1. A view is a named virtual table defined by a SELECT query that is stored in the database and can be queried like a regular table, abstracting the underlying query complexity.

2. Views simplify complex queries by encapsulating joins, filters, and aggregations behind a clean interface; they also improve security by restricting column/row visibility to specific users.

3. MySQL supports updatable views (allowing INSERT/UPDATE/DELETE on single-table views without aggregates or DISTINCT) and non-updatable views for complex multi-table or aggregated queries.

Q116. What is the difference between DELETE, TRUNCATE, and DROP?

1. DELETE removes specific rows matching a WHERE clause (or all rows if omitted), logs each deletion in the transaction log, and can be rolled back within a transaction.

2. TRUNCATE removes all rows from a table instantly by deallocating data pages, resets AUTO_INCREMENT counters, cannot be filtered by WHERE, and is not transaction-safe in MySQL.

3. DROP permanently removes the entire table definition and all its data, indexes, triggers, and constraints from the database; it cannot be rolled back and requires DDL privileges.

Q117. What is an AUTO_INCREMENT column in MySQL?

1. AUTO_INCREMENT is a column attribute that automatically generates a unique, incrementing integer value for each new row inserted, typically used for primary key columns.

2. MySQL maintains a per-table AUTO_INCREMENT counter that increments by 1 (configurable via auto_increment_increment) with each successful insert; the counter never decreases even after row deletions.

3. In InnoDB, the AUTO_INCREMENT counter is stored in memory in MySQL 5.7 and earlier (reset on restart if rows are deleted) but is persisted to the redo log in MySQL 8.0+, preventing gaps after restarts.

Q118. What is the difference between CHAR and VARCHAR in MySQL?

1. CHAR(n) stores fixed-length strings, always using exactly n bytes (padded with spaces if shorter), providing predictable storage and fast retrieval for fixed-size values like country codes or status flags.

2. VARCHAR(n) stores variable-length strings using only the actual character length plus 1-2 bytes for the length prefix, saving storage for columns with widely varying lengths.

3. CHAR is marginally faster for fixed-length data on row comparisons; VARCHAR is more space-efficient for variable data; both have a maximum of 65,535 bytes per row in InnoDB's row format.

Q119. What are MySQL data types for large text and binary data?

1. TEXT types (TINYTEXT, TEXT, MEDIUMTEXT, LONGTEXT) store variable-length character data up to 4GB; they are stored off-page for large values, preventing bloated row sizes.

2. BLOB types (TINYBLOB, BLOB, MEDIUMBLOB, LONGBLOB) store raw binary data such as images, documents, or serialised objects; their size limits mirror the TEXT family.

3. Storing large binary files directly in MySQL is generally discouraged for scalability reasons; a common pattern is storing files in object storage (S3, GCS) and saving only the file URL in MySQL.

Q120. What is the EXPLAIN command in MySQL?

1. EXPLAIN prepends to a SELECT (or DML) statement to output the query execution plan, showing how MySQL's query optimiser intends to access tables, use indexes, and join results.

2. Key columns in EXPLAIN output include type (access method: ALL for full scan, ref/eq_ref for indexed access), key (index used), rows (estimated rows examined), and Extra (additional information like 'Using filesort' or 'Using temporary').

3. Analysing EXPLAIN output is the first step in query optimisation; identifying full table scans (type: ALL) or 'Using filesort' indicates opportunities to add or improve indexes.

Q121. What is query optimisation in MySQL?

1. MySQL's query optimiser automatically analyses available indexes, table statistics, and join orders to generate the most efficient execution plan for each query.

2. Optimisation techniques include adding appropriate indexes on WHERE/JOIN/ORDER BY columns, rewriting correlated subqueries as JOINs, avoiding functions on indexed columns in WHERE clauses, and using query caching.

3. EXPLAIN ANALYZE (MySQL 8.0.18+) executes the query and reports actual vs estimated row counts and execution times, providing precise data for identifying and resolving optimisation opportunities.

Q122. What is a composite index in MySQL?

1. A composite (multi-column) index is an index defined on two or more columns, used by MySQL when query conditions reference the leading prefix of the indexed columns.

2. MySQL follows the leftmost prefix rule: a composite index on (a, b, c) can be used for queries filtering on a, a+b, or a+b+c, but not on b alone or c alone.

3. Column order in a composite index should place the most selective and frequently filtered column first; the index should be designed to cover the most common query patterns on the table.

Q123. What is a covering index in MySQL?

1. A covering index is an index that contains all columns referenced in a query (SELECT list, WHERE clause, JOIN conditions), allowing MySQL to satisfy the query entirely from the index without accessing the actual table rows.

2. Covering indexes eliminate the secondary lookup (index row to table row) for InnoDB secondary indexes, significantly reducing I/O for read-heavy queries on large tables.

3. EXPLAIN output shows 'Using index' in the Extra column when a covering index is used, indicating the query is served entirely from the index—one of the most impactful single-query optimisations available.

Q124. What are MySQL's JOIN types?

1. INNER JOIN returns rows where the join condition matches in both tables; OUTER JOINs (LEFT, RIGHT, FULL OUTER) return unmatched rows from one or both tables with NULLs for missing data.

2. CROSS JOIN produces a Cartesian product of two tables, returning every combination of rows; it is used intentionally for generating combinations but must be used carefully due to exponential row growth.

3. SELF JOIN joins a table to itself using aliases, useful for hierarchical data (e.g., employee-manager relationships) or comparing rows within the same table.

Q125. What is the difference between UNION and UNION ALL?

1. UNION combines the result sets of two or more SELECT statements, automatically removing duplicate rows using a sort-and-dedup operation.

2. UNION ALL returns all rows from all SELECT statements including duplicates, avoiding the deduplication overhead and is therefore significantly faster than UNION.

3. UNION ALL should be used whenever duplicates are either acceptable or impossible (e.g., unioning non-overlapping date ranges); UNION is appropriate only when deduplication is genuinely required.

Q126. What is MySQL replication?

1. MySQL replication is the process of automatically copying data changes from a source (primary) server to one or more replica (secondary) servers, providing high availability and read scalability.

2. The primary logs changes to the binary log (binlog); replicas connect, read the binlog events, and apply them via the replica SQL thread, maintaining an eventually consistent copy of the primary's data.

3. MySQL supports statement-based (SBR), row-based (RBR), and mixed replication; row-based replication is the safest and most commonly used mode as it replicates actual row changes rather than SQL statements.

Q127. What is MySQL's binary log?

1. The binary log (binlog) is a sequential log file that records all DDL and DML statements (or row change events in RBR mode) that modify database data.

2. Binlog is used for point-in-time recovery (restoring a backup and replaying binlog events to recover to a specific moment) and for replication between primary and replica servers.

3. Binary logging has a performance overhead; row-based binary logging generates larger log files than statement-based but is more reliable for complex queries involving non-deterministic functions.

Q128. What is InnoDB's MVCC?

1. Multi-Version Concurrency Control (MVCC) is InnoDB's mechanism for providing consistent reads without row-level read locks by maintaining multiple versions of each row.

2. When a transaction reads a row, InnoDB constructs the row's version as it existed at the start of the transaction using undo log records, ensuring consistent snapshot reads.

3. MVCC enables high read concurrency because readers do not block writers and writers do not block readers, allowing InnoDB to achieve the REPEATABLE READ isolation level with minimal locking overhead.

Q129. What is the InnoDB buffer pool?

1. The InnoDB buffer pool is the primary in-memory cache for InnoDB data pages (table rows, index pages), designed to absorb the majority of database I/O.

2. MySQL uses an LRU variant (midpoint insertion strategy) to manage the buffer pool, keeping frequently accessed 'hot' pages in memory and evicting cold pages when space is needed.

3. Sizing the buffer pool (innodb_buffer_pool_size) is the single most impactful MySQL configuration parameter; it should be set to 70-80% of total system RAM for dedicated database servers.

Q130. What is partitioning in MySQL?

1. Table partitioning divides a large table into smaller, independently managed segments (partitions) stored separately on disk, improving query performance through partition pruning.

2. MySQL supports RANGE, LIST, HASH, and KEY partitioning; RANGE partitioning is most common for time-series data, allowing queries to scan only the relevant date range partitions.

3. Partition pruning enables MySQL to skip partitions that cannot contain matching rows, dramatically reducing I/O for large tables; however, partitioning adds DDL complexity and has limitations with foreign keys.

Q131. What is MySQL's query cache?

1. MySQL's query cache (removed in MySQL 8.0) stored the complete result set of SELECT queries in memory, returning cached results for identical queries without re-execution.

2. The query cache was deprecated because it caused severe mutex contention under high write loads—any write to a table invalidated all cached queries for that table, negating the benefit.

3. Modern MySQL performance strategies use ProxySQL query caching, application-level caching (Redis, Memcached), and proper indexing rather than MySQL's built-in query cache.

Q132. What are window functions in MySQL?

1. Window functions (introduced in MySQL 8.0) perform calculations across a set of rows related to the current row (a 'window') without collapsing rows like GROUP BY aggregations.

2. Common window functions include ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD(), SUM() OVER, and AVG() OVER; they are defined using the OVER clause with optional PARTITION BY and ORDER BY.

3. Window functions enable complex analytical queries such as running totals, moving averages, percentile rankings, and lead/lag comparisons that previously required self-joins or application-level processing.

Q133. What is a CTE (Common Table Expression) in MySQL?

1. A CTE (introduced in MySQL 8.0) is a named temporary result set defined with the WITH keyword that exists only within the scope of the SQL statement that defines it.

2. CTEs improve query readability by breaking complex queries into named logical steps; they also enable recursive queries (WITH RECURSIVE) for traversing hierarchical data like trees and graphs.

3. Unlike subqueries, CTEs can be referenced multiple times within the same query, and recursive CTEs unlock hierarchical traversal patterns that are otherwise impossible or highly complex in standard SQL.

Q134. What is the difference between a clustered and non-clustered index in MySQL?

1. A clustered index determines the physical storage order of table rows on disk; InnoDB always uses the primary key as the clustered index, meaning table data is stored sorted by the primary key.

2. A non-clustered (secondary) index is a separate B-Tree structure where leaf nodes store the indexed column values and the corresponding primary key values used to look up the full row.

3. Secondary index lookups in InnoDB require two B-Tree traversals: one to find the primary key from the secondary index and one to retrieve the row from the clustered index, unless a covering index is used.

Q135. What is MySQL's EXPLAIN ANALYZE?

1. EXPLAIN ANALYZE (MySQL 8.0.18+) is an extended version of EXPLAIN that actually executes the query and reports both the estimated execution plan and the actual runtime statistics for each step.

2. It reports actual vs estimated row counts and actual execution time per node, enabling precise identification of cardinality estimation errors that mislead the query optimiser.

3. EXPLAIN ANALYZE is the most powerful built-in query debugging tool in MySQL; the gap between estimated and actual rows is the primary signal for missing or stale index statistics.

Q136. What is MySQL's slow query log?

1. The slow query log is a MySQL diagnostic feature that records queries exceeding a configurable execution time threshold (long_query_time), enabling systematic identification of performance bottlenecks.

2. It can be configured to log queries not using indexes (log_queries_not_using_indexes), capturing common sources of degraded performance even for fast-executing full table scans.

3. The mysqldumpslow tool and Percona's pt-query-digest parse and aggregate slow query log entries by query pattern, providing actionable summaries of the most impactful queries to optimise.

Q137. What is MySQL's full-text search?

1. MySQL full-text search allows efficient natural language searching within TEXT and VARCHAR columns using MATCH() ... AGAINST() syntax, rather than slower LIKE '%term%' pattern matching.

2. Full-text indexes can be created on MyISAM and InnoDB (MySQL 5.6+) tables; they tokenise text into words, build an inverted index, and support natural language mode and boolean mode queries.

3. Full-text search in MySQL is suitable for basic text search needs; for advanced requirements (relevance tuning, multi-language support, aggregations), dedicated search engines like Elasticsearch or Typesense are preferred.

Q138. What are MySQL's JSON data type and functions?

1. MySQL 5.7+ includes a native JSON data type that validates JSON syntax on insert, stores JSON in a binary format optimised for efficient key lookup without reparsing, and rejects invalid JSON.

2. JSON functions like JSON_EXTRACT (or the ->> shorthand), JSON_OBJECT, JSON_ARRAY, JSON_SET, JSON_REMOVE, and JSON_CONTAINS enable querying and modifying JSON documents with SQL.

3. Functional indexes on JSON expressions (MySQL 8.0+) enable B-Tree indexing of specific JSON paths, making filtered queries on JSON attributes as fast as queries on indexed relational columns.

Q139. What is MySQL Workbench?

1. MySQL Workbench is the official visual database design, administration, and query tool provided by Oracle for MySQL, available as a free cross-platform desktop application.

2. It provides an EER (Enhanced Entity-Relationship) diagram designer for visual schema modelling, a SQL editor with syntax highlighting and autocomplete, and a server administration interface for performance monitoring.

3. Workbench's Migration Wizard supports migrating schemas and data from other RDBMS platforms (SQL Server, PostgreSQL, Oracle) to MySQL, automating syntax conversion and data transfer.

Q140. What is database sharding in the context of MySQL?

1. Sharding is a horizontal scaling technique that partitions a database into independent smaller databases (shards) distributed across multiple servers, with each shard containing a subset of the data.

2. A shard key (e.g., user_id % N) determines which shard holds each record; the application layer or a proxy (Vitess, ProxySQL) routes queries to the appropriate shard.

3. Sharding dramatically increases write throughput and storage capacity but introduces significant complexity: cross-shard joins and transactions are expensive, and resharding requires careful data migration planning.

Q141. What is Vitess?

1. Vitess is an open-source database clustering system for MySQL developed at YouTube that provides horizontal sharding, connection pooling, query routing, and online schema changes.

2. It adds a lightweight proxy layer (vtgate) in front of MySQL that transparently handles shard routing, query rewriting, and resharding without requiring application changes.

3. Vitess is used at YouTube, Slack, and GitHub Scale, proving its ability to manage millions of QPS across thousands of MySQL instances while maintaining a familiar MySQL protocol for applications.

Q142. What is connection pooling in MySQL?

1. Connection pooling maintains a cache of reusable database connections so that applications can reuse existing connections rather than incurring the overhead of establishing a new TCP connection and authentication for each query.

2. MySQL has a per-connection overhead (thread creation, authentication, memory allocation) that makes creating new connections expensive; a pool like PgBouncer (for MySQL: ProxySQL or HikariCP) amortises

this cost.

3. Pool size should be tuned to match the database server's capacity: too few connections cause queueing; too many connections exhaust server memory and compete for resources, degrading performance.

Q143. What is the difference between MyISAM and InnoDB?

1. InnoDB supports ACID transactions, foreign keys, row-level locking, MVCC, and crash recovery via its redo/undo log infrastructure; MyISAM supports none of these.

2. MyISAM uses table-level locking, making it unsuitable for concurrent write workloads; InnoDB's row-level locking allows high write concurrency with minimal contention.

3. MyISAM is generally only considered today for specialised read-only or full-text search scenarios on very old MySQL versions; InnoDB is the correct default for virtually all modern MySQL workloads.

Q144. What is a deadlock in MySQL?

1. A deadlock occurs when two or more transactions are each waiting for a lock held by the other, creating a circular dependency that prevents any transaction from proceeding.

2. InnoDB automatically detects deadlocks and resolves them by rolling back the transaction with the lowest deadlock weight (typically the one that has modified fewer rows), allowing the other to proceed.

3. Deadlocks can be minimised by always acquiring locks in a consistent order across transactions, keeping transactions short, and using appropriate index coverage to reduce lock scope.

Q145. What are MySQL's locking types?

1. Table-level locks (LOCK TABLES READ/WRITE) lock the entire table, preventing concurrent modifications; they are used by MyISAM and for specific maintenance operations like ALTER TABLE in older MySQL versions.

2. Row-level locks (used by InnoDB) lock only the specific rows being read or modified, allowing concurrent access to different rows in the same table, dramatically improving concurrency.

3. Gap locks and next-key locks (InnoDB) lock the gap between index records to prevent phantom reads in REPEATABLE READ isolation, ensuring that range scans return consistent results across a transaction.

Q146. What is the difference between optimistic and pessimistic locking in MySQL?

1. Pessimistic locking acquires an exclusive lock on a row before reading it (SELECT ... FOR UPDATE) to prevent other transactions from modifying it before the current transaction completes.

2. Optimistic locking avoids database-level locks by adding a version column to rows; an UPDATE succeeds only if the version matches what was read, and the application retries if a concurrent modification is detected.

3. Pessimistic locking is appropriate for high-conflict scenarios where concurrent modifications are frequent; optimistic locking is more scalable in low-conflict scenarios, reducing lock contention at the cost of retry logic.

Q147. What is MySQL's redo log?

1. The InnoDB redo log is a write-ahead log (WAL) that records all changes to data pages before they are written to the actual tablespace files, ensuring durability and enabling crash recovery.

2. On crash, InnoDB replays redo log entries to recover committed transactions whose data pages had not yet been flushed to disk, restoring the database to a consistent state.

3. The redo log consists of circular log files (innodb_log_file_size); larger log files improve write performance by reducing checkpoint frequency but increase crash recovery time.

Q148. What is MySQL's undo log?

1. The InnoDB undo log stores the old versions of modified rows, enabling MVCC (consistent reads for concurrent transactions) and transactional rollback to undo uncommitted changes.

2. Undo records allow InnoDB to reconstruct the row's previous state for readers that need a consistent snapshot predating the current transaction's modifications.

3. Long-running transactions accumulate large amounts of undo data, growing the undo tablespace and slowing purge operations; they should be avoided or committed frequently in high-write environments.

Q149. What is MySQL's general query log?

1. The general query log records every SQL statement received by the MySQL server (connections, queries, disconnections) with a timestamp, providing a complete audit trail of all database activity.

2. It is an invaluable debugging tool for identifying unexpected queries, auditing user activity, or reproducing issues, but generates extremely high I/O volume and is unsuitable for continuous production use.

3. The general query log should be enabled temporarily for debugging sessions or directed to a table (log_output=TABLE) to query it with SQL; it must be disabled after use to prevent performance degradation.

Q150. What is an ORM and how does it interact with MySQL?

1. An Object-Relational Mapper (ORM) maps database tables to application objects and provides an API for querying and manipulating data using the application's native programming language rather than raw SQL.

2. Popular Node.js ORMs for MySQL include Sequelize, TypeORM, and Prisma; they handle connection management, query building, schema migrations, and relationships via model definitions.

3. ORMs accelerate development and reduce SQL injection risk via parameterised queries, but can generate inefficient SQL for complex queries; developers should use raw queries or query builders for performance-critical paths.

Q151. What is MySQL's character set and collation?

1. A character set defines the set of characters that can be stored (e.g., utf8mb4 supports all Unicode characters including emoji); a collation defines the rules for comparing and sorting characters.

2. utf8mb4 with utf8mb4_unicode_ci is the recommended default for MySQL databases storing international text, as utf8 (3-byte) cannot store 4-byte Unicode characters like emoji.

3. Character set mismatches between client, connection, and server can cause encoding corruption; setting character_set_client, character_set_connection, and character_set_results to utf8mb4 ensures consistent encoding.

Q152. What is MySQL's DATETIME vs TIMESTAMP?

1. DATETIME stores a date and time value as-is with no time zone conversion, ranging from '1000-01-01 00:00:00' to '9999-12-31 23:59:59', suitable for representing times independent of time zone.

2. TIMESTAMP stores values as UTC internally and converts to/from the server's time zone on read/write, ranging from '1970-01-01 00:00:01' UTC to '2038-01-19 03:14:07' UTC (Y2K38 limitation).

3. TIMESTAMP should be used for event timestamps that must be consistent across time zones; DATETIME is appropriate for calendar values (e.g., appointment times) that should not be time zone converted.

Q153. What is the INFORMATION_SCHEMA in MySQL?

1. INFORMATION_SCHEMA is a virtual database containing read-only system views that expose metadata about all other databases, tables, columns, indexes, users, and privileges on the MySQL server.

2. Commonly queried views include TABLES (table metadata), COLUMNS (column definitions), KEY_COLUMN_USAGE (foreign key relationships), and PROCESSLIST (active connections and queries).

3. INFORMATION_SCHEMA queries are widely used in database administration scripts for schema inspection, dynamic SQL generation, monitoring, and automating database management tasks.

Q154. What is MySQL's performance_schema?

1. The Performance Schema is an instrumentation framework in MySQL that collects detailed runtime metrics about server execution, including query execution time, wait events, memory usage, and I/O statistics.

2. It exposes its data through tables in the performance_schema database, enabling SQL queries to identify slow queries, contended mutexes, high-latency I/O, and memory consumers.

3. The sys schema (MySQL 5.7.7+) provides a set of views and stored procedures built on performance_schema that simplify common DBA diagnostic queries into human-readable formats.

Q155. What are prepared statements in MySQL?

1. Prepared statements separate the SQL query structure from data values; the query is compiled once by the MySQL server and executed multiple times with different bound parameter values.

2. They prevent SQL injection by ensuring user-supplied data is always treated as a parameter value rather than part of the SQL syntax, regardless of special characters it may contain.

3. Prepared statements improve performance for repeatedly executed queries with different parameters by caching the query execution plan, reducing parsing and optimisation overhead on subsequent executions.

Q156. What is SQL injection and how does MySQL prevent it?

1. SQL injection is an attack where malicious SQL code is inserted into an input parameter that is concatenated into a SQL query, potentially manipulating the query to expose, modify, or delete data.

2. MySQL prevents SQL injection through prepared statements and parameterised queries, which bind user input as typed data values rather than interpolating them into the SQL string.

3. Additional defences include least-privilege database users (granting only necessary permissions), input validation at the application layer, and web application firewalls (WAFs) that detect SQL injection patterns.

Q157. What is the difference between TRUNCATE and DELETE in MySQL regarding transactions?

1. DELETE is a DML statement that removes rows one at a time, generates undo log entries for each deleted row, acquires row locks, and can be rolled back within an open transaction.

2. TRUNCATE is a DDL statement that deallocates data pages atomically, does not generate per-row undo log entries, and implicitly commits any open transaction before executing in MySQL.

3. Because TRUNCATE commits the current transaction, it cannot be rolled back in MySQL (unlike in PostgreSQL where DDL is transactional); applications must account for this when using TRUNCATE in transactional workflows.

Q158. What is MySQL's GROUP BY clause?

1. GROUP BY collapses multiple rows with the same value(s) in the specified column(s) into a single summary row, enabling aggregate functions (SUM, COUNT, AVG, MIN, MAX) to operate on each group.

2. In MySQL 5.7+, ONLY_FULL_GROUP_BY SQL mode (enabled by default) requires that all non-aggregated SELECT columns appear in the GROUP BY clause, preventing non-deterministic query results.

3. GROUP BY is foundational for reporting and analytics queries; optimising its performance involves ensuring the grouping columns are indexed and the result set size is managed through appropriate WHERE filtering.

Q159. What are MySQL's aggregate functions?

1. COUNT(expr) counts non-NULL values or rows; COUNT(*) counts all rows including NULLs; COUNT(DISTINCT col) counts unique non-NULL values.

2. SUM(col) computes the total of numeric values; AVG(col) computes the mean; MIN(col) and MAX(col) return the smallest and largest values in the group.

3. GROUP_CONCAT(col ORDER BY col SEPARATOR ',') aggregates string values from multiple rows into a single delimited string, useful for creating comma-separated lists in SQL results.

Q160. What is the purpose of the LIMIT clause in MySQL?

1. LIMIT restricts the number of rows returned by a SELECT query, essential for pagination, sampling, and preventing accidentally large result sets from overwhelming application memory.

2. LIMIT n OFFSET m skips m rows before returning n rows, enabling cursor-based pagination; however, large OFFSET values are inefficient as MySQL must scan and discard all skipped rows.

3. Keyset pagination (WHERE id > last_seen_id LIMIT n) is the recommended alternative to OFFSET-based pagination for high-page-number access on large tables, as it uses an index seek rather than a full scan.

Q161. What is MySQL's ORDER BY clause and its performance implications?

1. ORDER BY sorts the result set by one or more columns in ASC (default) or DESC order; without an ORDER BY, row order in MySQL is non-deterministic and should never be relied upon.

2. If the ORDER BY columns match the prefix of an existing index and the query can be resolved using that index, MySQL avoids a costly filesort operation (visible as 'Using filesort' in EXPLAIN).

3. For large result sets requiring sort, adding a composite index on (WHERE columns, ORDER BY columns) is the most effective optimisation, allowing MySQL to return pre-sorted results directly from the index.

Q162. What is a MySQL schema migration?

1. A schema migration is a versioned, incremental change to a database schema (adding columns, creating indexes, modifying data types) managed as code to track database evolution alongside application code.

2. Migration tools like Flyway, Liquibase, and Prisma Migrate maintain a migrations table recording which migrations have been applied, enabling safe, repeatable deployments across environments.

3. Online schema changes (Percona's pt-online-schema-change or MySQL 8.0's instant ADD COLUMN) minimise table locking during production ALTER TABLE operations, reducing downtime for large table migrations.

Q163. What are the GRANT and REVOKE statements in MySQL?

1. GRANT assigns specific privileges (SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, etc.) to a MySQL user on a specific database, table, or column level.

2. REVOKE removes previously granted privileges from a user; changes take effect immediately for new connections and can be applied to the current session with FLUSH PRIVILEGES.

3. Following the principle of least privilege, application users should be granted only the specific privileges required for their function (e.g., SELECT/INSERT/UPDATE on specific tables), never SUPER or GRANT OPTION.

Q164. What is the MySQL SHOW commands collection?

1. SHOW DATABASES lists all databases; SHOW TABLES lists tables in the current database; SHOW COLUMNS FROM table describes column definitions; SHOW INDEX FROM table displays index details.

2. SHOW CREATE TABLE produces the complete DDL statement to recreate a table, including all column definitions, constraints, and index definitions, essential for schema documentation and migration.

3. SHOW PROCESSLIST displays currently executing connections and queries with their state and elapsed time, enabling real-time identification of long-running queries, locks, and connection floods.

Q165. What is MySQL Group Replication?

1. MySQL Group Replication (MGR) is a fault-tolerant replication topology where multiple MySQL servers form a consensus group using the Paxos protocol, enabling automatic failover and multi-primary writes.

2. In single-primary mode, one node accepts writes and replicates to all others; in multi-primary mode, all nodes accept writes with conflict detection and resolution handled by the group protocol.

3. MGR provides high availability with automatic primary election on failure and is the foundation of MySQL InnoDB Cluster, which adds the MySQL Shell and MySQL Router for a complete HA solution.

Q166. What is MySQL InnoDB Cluster?

1. MySQL InnoDB Cluster is a complete high-availability solution combining Group Replication (consensus-based replication), MySQL Router (transparent failover routing), and MySQL Shell (administration interface).

2. MySQL Router sits between the application and the cluster, transparently routing read/write traffic to the current primary and distributing read traffic to secondaries, with automatic failover on primary failure.

3. InnoDB Cluster provides automated failover in seconds, requiring no manual intervention or application reconfiguration, making it a production-grade HA solution for MySQL deployments.

Q167. What is MySQL's LOAD DATA INFILE?

1. LOAD DATA INFILE is a highly optimised MySQL statement that bulk-loads data from a CSV or delimited text file directly into a table, bypassing the row-by-row INSERT overhead.

2. It is typically 10-100x faster than equivalent INSERT statements for large data loads because it batches I/O, minimises transaction log overhead, and uses optimised internal bulk loading paths.

3. Security restrictions require the file to be accessible on the server (LOAD DATA INFILE) or the client (LOAD DATA LOCAL INFILE), with appropriate file permissions and the local_infile variable enabled for the client variant.

Q168. What is MySQL's INSERT ... ON DUPLICATE KEY UPDATE?

1. INSERT ... ON DUPLICATE KEY UPDATE performs an INSERT, and if a unique key or primary key conflict is detected, executes the specified UPDATE on the conflicting existing row instead of failing.

2. This is an efficient upsert pattern for maintaining counters, timestamps, or configuration values without requiring a separate SELECT to check for existence before INSERT or UPDATE.

3. The statement affects 1 row for a successful INSERT, 2 rows for an UPDATE, and 0 rows if the ON DUPLICATE KEY UPDATE assigns identical values; the ROW_COUNT() function reflects this behaviour.

Q169. What is MySQL's REPLACE INTO?

1. REPLACE INTO works like INSERT but first DELETEs any existing row with a conflicting primary or unique key before inserting the new row, effectively replacing the old row.

2. Unlike ON DUPLICATE KEY UPDATE, REPLACE always performs a DELETE + INSERT, which resets AUTO_INCREMENT, resets all columns to defaults for unspecified columns, and triggers DELETE and INSERT triggers.

3. REPLACE INTO is generally less safe than INSERT ... ON DUPLICATE KEY UPDATE because the DELETE step can cascade to child tables via foreign keys and cause unexpected data loss.

Q170. What is MySQL's EXPLAIN FORMAT=JSON?

1. EXPLAIN FORMAT=JSON provides the query execution plan in JSON format with richer details than tabular EXPLAIN, including cost estimates, filtered row percentages, and nested join structures.

2. The JSON output includes used_key_parts, key_length, and attached_condition fields that are absent from tabular EXPLAIN, providing deeper insight into how MySQL evaluates complex WHERE conditions.

3. Tools like MySQL Workbench's Visual Explain use EXPLAIN FORMAT=JSON to generate graphical execution plan diagrams, making complex multi-join query plans easier to interpret visually.

Q171. What is MySQL's innodb_flush_log_at_trx_commit?

1. innodb_flush_log_at_trx_commit controls the durability vs performance trade-off for InnoDB's redo log flushing at transaction commit.

2. Value 1 (default, safest) flushes and syncs the log to disk on every commit, guaranteeing no data loss on OS or hardware crash; value 2 writes to OS cache on commit but syncs only once per second.

3. Value 0 (least durable) neither flushes nor syncs per commit, risking up to one second of committed transaction loss on crash; value 2 is a common performance compromise that risks only OS-crash data loss.

Q172. What is MySQL's sync_binlog?

1. sync_binlog controls how frequently MySQL synchronises the binary log to disk, balancing durability against write throughput.

2. sync_binlog=1 syncs the binlog to disk before each transaction commit, ensuring zero binlog data loss on crash at the cost of additional fsync operations per transaction.

3. For maximum durability, both innodb_flush_log_at_trx_commit=1 and sync_binlog=1 should be set together (the 'double-1' configuration); either set to a non-1 value introduces a data loss risk window.

Q173. What is Percona Server for MySQL?

1. Percona Server is an enhanced, fully compatible drop-in replacement for MySQL Community Edition that adds performance improvements, additional monitoring metrics, and enterprise features at no cost.

2. Key additions include XtraDB (an InnoDB fork with improved performance), Percona XtraBackup (hot backup tool), enhanced INFORMATION_SCHEMA tables, and improved thread pool implementation.

3. Percona Server is widely used in high-performance production environments where the additional diagnostics (QUERY_RESPONSE_TIME, USER_STATISTICS) and performance improvements justify the fork.

Q174. What is MySQL's GROUP_CONCAT function?

1. GROUP_CONCAT aggregates multiple row values from a group into a single comma-separated (or custom separator) string within a GROUP BY query.

2. It supports ORDER BY to sort values within the concatenation and DISTINCT to remove duplicates; the maximum length is controlled by the group_concat_max_len system variable (default 1024 bytes).

3. GROUP_CONCAT is widely used for pivoting relational data into comma-separated lists (e.g., all tags for an article) within a single query row without requiring multiple round trips.

Q175. What are MySQL's spatial data types and functions?

1. MySQL supports spatial data types (GEOMETRY, POINT, LINESTRING, POLYGON, MULTIPOINT, etc.) for storing geometric and geographic data per the OpenGIS specification.

2. Spatial functions like ST_Distance, ST_Within, ST_Intersects, ST_Contains, and ST_GeomFromText enable geometric calculations and spatial relationship queries within SQL.

3. Spatial indexes (SPATIAL INDEX on geometry columns) use R-Tree indexing to accelerate proximity and containment queries, enabling efficient geospatial searches like 'find all points within this polygon'.

Q176. What is the MySQL event scheduler?

1. The MySQL Event Scheduler is an internal task scheduler that executes SQL statements or stored procedures automatically at scheduled times, analogous to a database-level cron job.

2. Events are created with CREATE EVENT, specifying a schedule (once at a specific time or recurring with an interval) and the SQL body to execute.

3. The event scheduler is useful for automated maintenance tasks like purging old data, generating summary tables, or refreshing materialised view-like tables without requiring external scheduling infrastructure.

Q177. What is a materialised view in MySQL?

1. MySQL does not natively support materialised views (precomputed, stored query result tables like PostgreSQL); they must be simulated using regular tables that are periodically refreshed.

2. A common pattern is creating a summary table, populating it via a stored procedure or event, and refreshing it on a schedule or via triggers when source data changes.

3. For complex reporting queries on large datasets, materialised view simulation significantly reduces query time by precomputing expensive aggregations, at the cost of data staleness and additional storage.

Q178. What is MySQL's ANALYZE TABLE?

1. ANALYZE TABLE collects and stores key distribution statistics for a table's indexes, updating the statistics used by the query optimiser to estimate rows and choose execution plans.

2. Stale statistics (after large data insertions, deletions, or updates) cause the optimiser to choose suboptimal execution plans; running ANALYZE TABLE refreshes statistics without affecting data.

3. InnoDB automatically updates statistics when 10% of rows change (configurable via innodb_stats_auto_recalc), but manual ANALYZE TABLE may be needed after bulk operations to immediately update statistics.

Q179. What is MySQL's OPTIMIZE TABLE?

1. OPTIMIZE TABLE defragments a table and its indexes, reclaiming wasted space from deleted rows and compacting the index B-Tree structure, improving storage efficiency and read performance.

2. For InnoDB tables, OPTIMIZE TABLE is equivalent to ALTER TABLE ... FORCE, which rebuilds the table and its clustered index; this can be a lengthy, lock-intensive operation on large tables.

3. Percona XtraBackup and pt-online-schema-change provide non-blocking alternatives to OPTIMIZE TABLE for production environments where downtime during optimisation is unacceptable.

Q180. What is MySQL's CHECK TABLE?

1. CHECK TABLE verifies the integrity of a table and its indexes, identifying corruption, inconsistencies, or errors in the storage engine data structures.

2. For InnoDB tables, CHECK TABLE verifies index structure and performs basic consistency checks; for MyISAM, it performs more thorough checks including key tree validation.

3. Regular CHECK TABLE execution is part of a proactive database health monitoring strategy; the mysqlcheck utility automates this across all databases and is commonly scheduled as a maintenance task.

Q181. What is MySQL's REPAIR TABLE?

1. REPAIR TABLE attempts to fix a corrupted MyISAM or ARCHIVE table by rebuilding the index file and correcting data inconsistencies detected by CHECK TABLE.

2. InnoDB tables generally do not support REPAIR TABLE; InnoDB corruption is addressed through crash recovery (InnoDB auto-recovery on restart) or restoring from backup.

3. Preventing corruption through proper hardware (ECC RAM, RAID with write-back cache), regular backups, and proper server shutdown procedures is far preferable to repair operations.

Q182. What is MySQL's binlog_format?

1. binlog_format controls how MySQL records data changes in the binary log: STATEMENT records the actual SQL statements; ROW records the before and after images of each modified row.

2. MIXED mode uses STATEMENT for safe statements and automatically switches to ROW for statements that could produce different results on replica (non-deterministic functions, triggers, etc.).

3. ROW-based binary logging is the recommended format for production replication as it guarantees replica data consistency regardless of non-deterministic functions or server configuration differences.

Q183. What is Point-in-Time Recovery (PITR) in MySQL?

1. PITR is a disaster recovery technique that restores a MySQL database to a specific moment by restoring the most recent full backup and then replaying binary log events up to the target time.

2. The process involves: restore the full/incremental backup, identify the target position in the binary log using mysqlbinlog, and apply all events up to but not including the undesired transaction.

3. PITR is essential for recovering from logical errors (accidental DROP TABLE, erroneous bulk updates) that are not addressed by replica failover or crash recovery alone.

Q184. What is Percona XtraBackup?

1. Percona XtraBackup is an open-source hot backup tool for MySQL and Percona Server that performs non-blocking physical backups of InnoDB tables without acquiring table locks.

2. It works by copying InnoDB data files while tracking ongoing changes in the redo log; the prepare phase then applies these redo log changes to produce a consistent backup snapshot.

3. XtraBackup supports incremental backups (capturing only pages changed since the last backup) and streaming backups to remote storage, making it the industry standard for MySQL backup in production environments.

Q185. What is MySQL's innodb_file_per_table?

1. innodb_file_per_table (enabled by default since MySQL 5.6) stores each InnoDB table's data and index in its own dedicated .ibd tablespace file rather than the shared system tablespace.

2. Per-table tablespace files enable space reclamation after DROP TABLE or TRUNCATE, support individual table OPTIMIZE TABLE operations, and allow transport of tables between servers via EXPORT/IMPORT.

3. The shared system tablespace (ibdata1) still stores the data dictionary (pre-8.0), undo logs, and double-write buffer; innodb_file_per_table only affects table data and index storage.

Q186. What is MySQL's doublewrite buffer?

1. The InnoDB doublewrite buffer is a write-ahead mechanism that writes data pages to a dedicated area of the tablespace before writing them to their actual locations, protecting against partial page writes.

2. A partial page write occurs when MySQL writes an incomplete 16KB InnoDB page to disk (e.g., due to OS crash mid-write on systems with 4KB native page size), corrupting the page.

3. The doublewrite buffer ensures that even after a crash, InnoDB can recover a complete copy of the page from the doublewrite buffer if the actual page copy is corrupt.

Q187. What is the difference between READ LOCK and WRITE LOCK in MySQL?

1. A READ (shared) lock allows multiple transactions to read a resource concurrently but prevents any transaction from acquiring a WRITE lock on it while the READ lock is held.

2. A WRITE (exclusive) lock prevents all other transactions from reading or writing the locked resource until the WRITE lock is released, ensuring exclusive access during modification.

3. InnoDB row-level locking uses shared (S) and exclusive (X) locks; intention locks (IS, IX) at the table level signal the type of row-level lock held, enabling efficient table-lock conflict detection.

Q188. What are MySQL's string functions?

1. Common string manipulation functions include CONCAT (concatenation), SUBSTRING (extract substring), REPLACE (find and replace), TRIM/LTRIM/RTRIM (whitespace removal), UPPER/LOWER (case conversion), and LENGTH/CHAR_LENGTH (byte vs character length).

2. Pattern matching functions include LIKE (wildcard: % and _), REGEXP/RLIKE (POSIX regular expressions in MySQL 5.7, PCRE-compatible in MySQL 8.0), and LOCATE/INSTR (substring position).

3. FORMAT(number, decimal_places) formats numbers with thousands separators; LPAD/RPAD pad strings to a fixed length; LEFT/RIGHT extract characters from string ends—all essential for formatting query output.

Q189. What are MySQL's date and time functions?

1. NOW() returns the current date and time as DATETIME; CURDATE() returns the current date; CURTIME() returns the current time; UTC_TIMESTAMP() returns the current UTC date and time.

2. DATE_ADD/DATE_SUB add or subtract time intervals (YEAR, MONTH, DAY, HOUR, MINUTE, SECOND); DATEDIFF calculates the number of days between two dates; TIMESTAMPDIFF computes differences in any unit.

3. DATE_FORMAT(date, format) formats a date/time value using format specifiers (%Y=4-digit year, %m=month, %d=day, %H=hour, %i=minute, %s=second), enabling flexible date display formatting in SQL.

Q190. What is MySQL's COALESCE function?

1. COALESCE(expr1, expr2, ..., exprN) returns the first non-NULL expression in its argument list, providing a concise way to substitute a default value for NULL in query results.

2. It is used extensively to handle nullable columns in reports (e.g., COALESCE(discount, 0) to treat NULL discount as zero in price calculations) and to cascade through multiple fallback values.

3. COALESCE is ANSI SQL standard and preferred over MySQL's proprietary IFNULL(expr, default) function for portability, though IFNULL is equivalent for the two-argument case.

Q191. What is MySQL's CASE expression?

1. The CASE expression provides conditional branching within SQL, returning different values based on conditions: CASE WHEN condition THEN result [WHEN ...] [ELSE default] END.

2. It can be used as a simple CASE (comparing an expression to values) or a searched CASE (evaluating independent boolean conditions), analogous to a switch/case or if/else in application code.

3. CASE is widely used in SELECT lists for conditional column transformations, in ORDER BY for custom sort logic, in aggregate functions for conditional counting/summing, and in UPDATE SET clauses.

Q192. What is MySQL's REGEXP function?

1. REGEXP (or RLIKE) matches a string against a regular expression pattern, returning 1 if the string matches and 0 otherwise, enabling powerful pattern-based row filtering.

2. MySQL 8.0 uses ICU REGEXP library supporting POSIX and Perl-compatible regular expressions (PCRE) with Unicode support; MySQL 5.7 and earlier used the POSIX regex library with limited features.

3. REGEXP cannot use standard B-Tree indexes for pattern matching (similar to LIKE with a leading wildcard), so it performs full table or full index scans; full-text search or application-layer filtering is preferred for performance.

Q193. What is MySQL's GENERATED column?

1. Generated columns (MySQL 5.7+) are columns whose values are derived from an expression based on other columns in the same row, defined using AS (expression) STORED or VIRTUAL.

2. VIRTUAL generated columns compute their value on read without storing data; STORED generated columns persist the computed value to disk, consuming storage but enabling indexing for fast access.

3. Stored generated columns can be indexed, enabling efficient queries on computed values (e.g., indexing the year extracted from a DATETIME column) without application-layer data duplication.

Q194. What is MySQL's CHECK constraint?

1. CHECK constraints (MySQL 8.0.16+) enforce domain integrity by specifying a Boolean expression that must evaluate to TRUE for every row INSERT or UPDATE operation on the table.

2. They validate business rules at the database level (e.g., CHECK (age >= 0 AND age <= 150), CHECK (status IN ('active','inactive'))), preventing invalid data regardless of the inserting application.

3. While MySQL 5.7 and earlier parsed CHECK syntax without enforcing it, MySQL 8.0+ enforces CHECK constraints, aligning with the SQL standard and providing database-level data quality guarantees.

Q195. What is MySQL's ENUM data type?

1. ENUM is a string data type where a column value must be chosen from a predefined list of values defined at table creation time; MySQL stores ENUM values internally as integers for efficiency.

2. ENUM provides implicit input validation (invalid values are rejected or converted to empty string depending on SQL mode), consistent string representation, and compact storage (1-2 bytes per row).

3. ENUM columns are problematic to modify (adding/reordering values requires an ALTER TABLE in older MySQL versions) and can cause application issues if the allowed values change; VARCHAR with a CHECK constraint is often more flexible.

Q196. What is MySQL's SET data type?

1. SET is a string data type that stores zero or more values from a predefined list, stored as a bit mask internally; unlike ENUM (single value), SET allows multiple values per cell.

2. SET is queried using FIND_IN_SET(value, set_column) or bitwise comparisons; it stores up to 64 members using 1-8 bytes depending on member count.

3. SET is rarely used in modern schema design; JSON arrays or a separate junction table provide more flexibility, maintainability, and query capability for multi-value attribute storage.

Q197. What is the difference between SIGNED and UNSIGNED integers in MySQL?

1. SIGNED integers can store both negative and positive values; INT SIGNED ranges from -2,147,483,648 to 2,147,483,647, with the sign bit using half the integer range.

2. UNSIGNED integers store only non-negative values, doubling the positive range: UNSIGNED INT ranges from 0 to 4,294,967,295, useful for primary keys, counters, and quantities that cannot be negative.

3. Using UNSIGNED for AUTO_INCREMENT primary keys is recommended to maximise the available key space and avoid premature overflow on high-volume tables before migrating to BIGINT.

Q198. What is MySQL's SPATIAL index?

1. A SPATIAL index is an R-Tree index optimised for geometric and geographic data types (GEOMETRY, POINT, POLYGON, etc.), enabling efficient spatial relationship queries.

2. SPATIAL indexes support bounding-box queries via functions like MBRContains, MBRWithin, and ST_Within, allowing MySQL to quickly identify candidate geometries before performing exact geometric tests.

3. SPATIAL indexes require NOT NULL geometry columns and are only supported by MyISAM and InnoDB (MySQL 5.7.5+); they are essential for location-based features like proximity searches and geographic boundary queries.

Q199. What is MySQL's HANDLER statement?

1. HANDLER is a low-level MySQL statement that provides direct access to InnoDB and MyISAM table storage engine interfaces, bypassing the SQL query layer for raw row-by-row access.

2. It supports sequential (HANDLER ... READ FIRST/NEXT/PREV/LAST) and indexed access (HANDLER ... READ index_name) operations with much lower overhead than equivalent SELECT statements.

3. HANDLER is rarely used in application code but is useful for specialised high-performance cursor iteration over large tables where minimising SQL parsing overhead matters, and for storage engine debugging.

Q200. What is MySQL's GTID replication?

1. Global Transaction Identifiers (GTIDs) are unique identifiers assigned to each committed transaction on the MySQL primary, replacing binlog file-position-based replication tracking.

2. GTIDs enable replicas to automatically determine which transactions they have applied and which they need to fetch, simplifying failover and replica provisioning by eliminating manual binlog position management.

3. GTID-based replication is strongly recommended for new MySQL deployments as it simplifies topology management, enables automatic failover tools (Orchestrator, MHA), and makes replica lag diagnosis more reliable.